Wrocław University of Science and Technology

# Artificial Intelligence & Knowledge Engineering

# Large Language Models

**Training and Prompt Engineering**

Maciej Piasecki

Katedra Sztucznej Inteligencji (Department of Artificial Intelligence)

CLARIN-PL

clarin-pl.eu

HR EXCELLENCE IN RESEARCH

This course was supported by NAWA STER Programme Internationalisation of Wroclaw University of Science and Technology Doctoral School

Wrocław University
of Science and Technology

# Large Language Model

- LLM = *Large Language Model*
  - Large generative, neural language model.
- Very large number of parameters:
  - from 1 billion (1B),
  - small: 7-20 billions (7B-20B),
  - medium: ~50 - 70 billions,
  - large: hundreds of billionss and more (>70B).
- Most is based on the decoder only architecture:
  - Autoregressive training: input text → input text
  - Parallel training on mass data

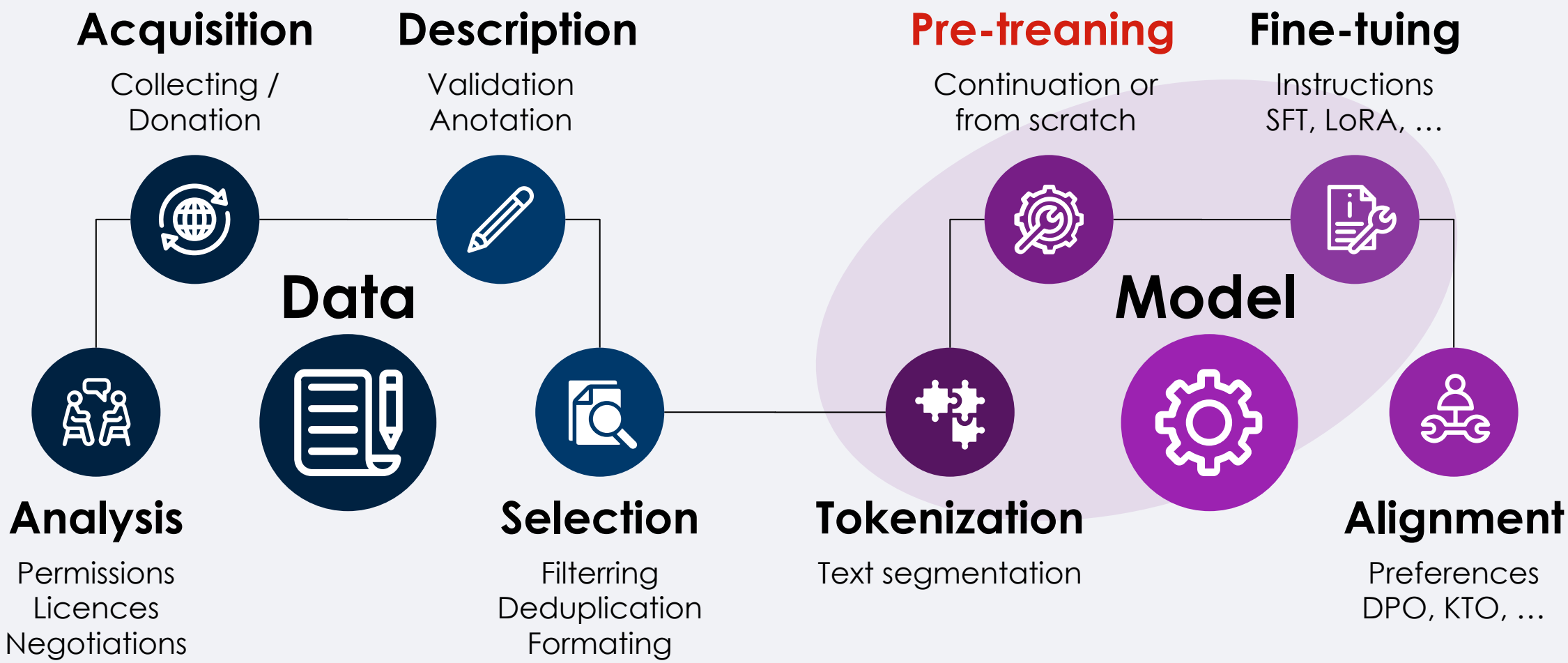
# Training LLMs

- Collecting a very large set of texts

1. Pretraining
   - **unsupervised** or **weakly supervised**
   - adaptation: language or domain – continual training
   - also *finetuning* or *language adaptation*.
2. Finetuning to the task
   - ***supervised*** *finetuning* (SFT), subtask (step) *instruction finetuning*.
3. Model alignment, especially to human preferences
   - **Reinforcement Learning**

- Yan LeCun's 'recipe' for future AI from 2013 r. (sic!)

# LLM Development Process



# Pretraining

- Unsupervised learning
- Autoregresive scheme:
  - Input text $T$ → model → regenerated text $\check{T}$
  - Error (loss): the difference between $\check{T}$ and $T$

# (1) Trening wstępny (ang. *pre-training*)

W pre-treningu uczymy model zadania *next-token prediction*. Maksymalizujemy log-prawdopodobieństwo prawidłowych tokenów w każdej pozycji sekwencji:

$$\log p_\theta(x) = \sum_{t=1}^{T} \log \pi_\theta(x_t \mid x_{<t})$$

Funkcja kosztu wykorzystywana w treningu wstępnym (minimalizowana):

$$\mathcal{L}_{\mathrm{NLL}}(\theta) = -\sum_{t=1}^{T} \log \pi_\theta(x_t \mid x_{<t})$$

Im więcej danych, tym bardziej ogólna wiedza i kompetencje!

# Generative basis for prompt engineering

- Input text:
  - Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relaxed. In short our stay was
- Potential continuations (e.g. GPT 3):
  - … not a pleasant one. The staff at the front desk were not welcoming or friendly, and seemed disinterested in providing good customer service.
  - … uncomfortable and not worth the price we paid. We will not be returning to this hotel.
  - correlation: sentiment of the continuation vs classification of the input text sentiment

# Fine-tuning

- Supervised training:
  - input: text of an instruction → model → result $\check{R}$
  - error: comparison of $\check{R}$ with $R$ (from training data)
- Instruction types:
  - Text conversion,
    - np. corrections, simplifications, style transformations, etc.
  - text fragment interpretation,
    - np. Named Entity Recognition, word grammatical form recognition,
  - text classification
    - e.g. sentiment, hate speech, thematic categories, etc.
  - Text interpretation,
    - E.g. keyword assignment,
  - Recognition of text-to-text relations,
    - implication, contradiction, overlap, attribution, etc.
  - Question Answering on the basis of texts,
  - Reaction on utterances in dialogue,
  - …

# (2) Dostrajanie (ang. Supervised Fine-tuning)

Model uczy się mapowania:

Instrukcja ⟶ Pożądane zachowanie

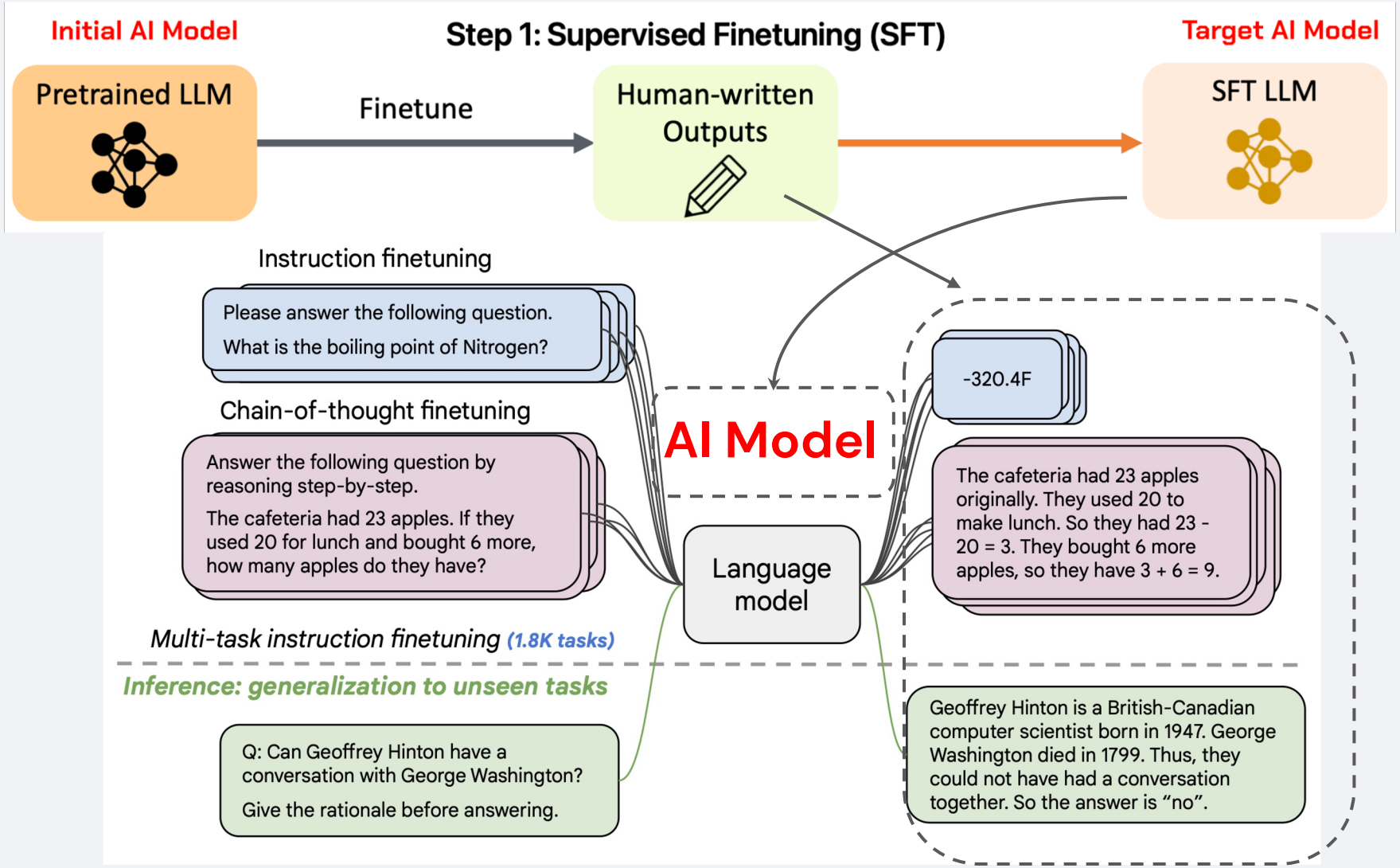
Osiągamy to poprzez minimalizację straty dla par ***(x, y)*** gdzie ***x*** to instrukcja, a ***y*** *to odpowiedź wzorcowa):*

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\sum_{t=1}^{T} \log \pi_\theta(y_t \mid y_{<t},\, x)$$

**Wymuszamy, aby model przypisywał coraz większe prawdopodobieństwo (kolejnym) tokenom odpowiedzi wzorcowej!**

Uwaga: Model otrzymuje zarówno instrukcję x jak i dotychczas wygenerowane tokeny i ma przewidzieć kolejny token odpowiedzi. Uczy się odtwarzać odpowiedź token po tokenie.

# (2) Dostrajanie (ang. Supervised Fine-tuning)

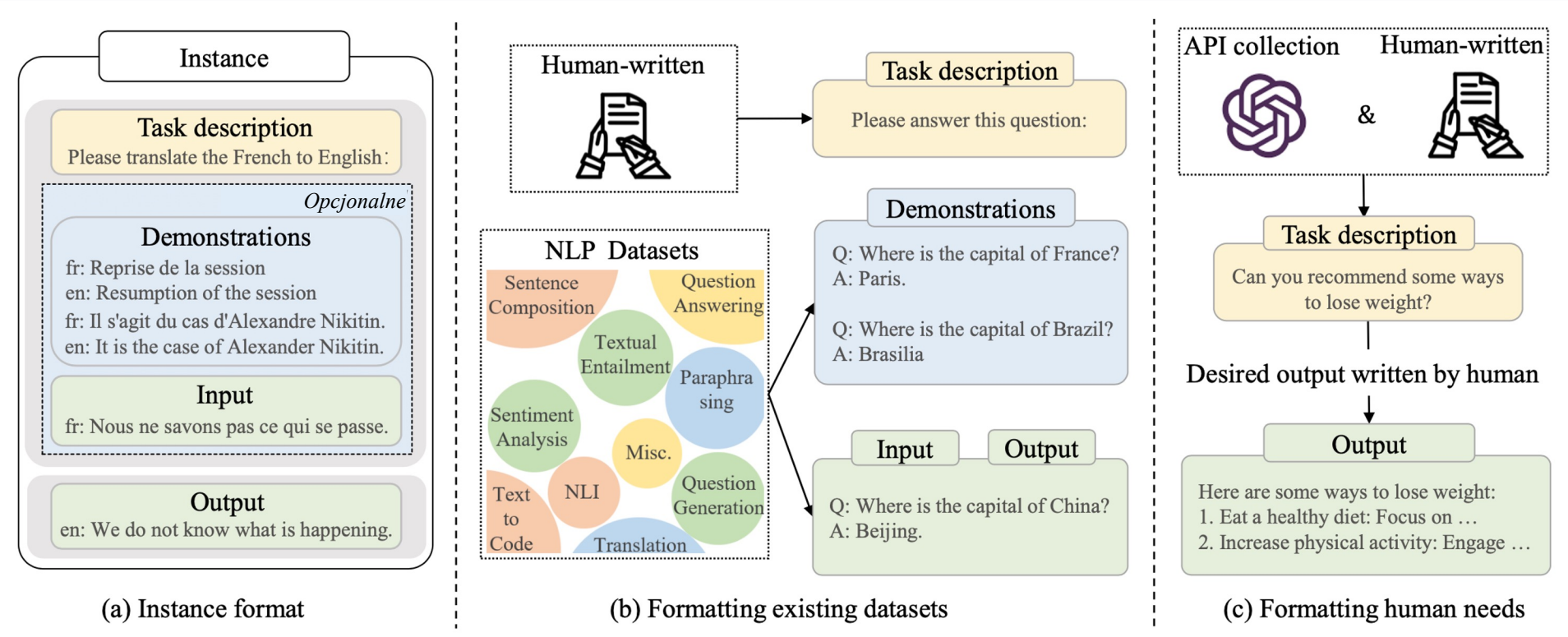



Source: HW Chung et al., *„Scaling Instruction-Finetuned Language Models"*

# Fine-tuning – an example instruction

- P: From the list of all the emotions provided, select those that the input text evokes in the majority of people reading it. Write down your answer in a Python list containing exactly the two most relevant emotions. A list of all emotions: admiration, amusement, anger, irritation, approval, concern, embarrassment, curiosity, desire, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, optimism, pride, realization, relief, remorse, sadness, surprise, neutral.
- Text: So, it seems that they are happy.
- R: [”excitement”, ”neutral”]

# Alignment




(Xin Zhao i in., 2023)

# Alignment

- Different methods of supervised machine learning
  - i.e. based on manually described examples (samples)
- Initially, reinforcement learning (e.g. ChatGPT) – key element: input correction
  1. The task is delivered to the model.
  2. The model generates a response.
  3. Response quality is manually evaluated → feedback (numerical).
  4. Feedback is used to direct the model towards better answers.
- Millions of repetitions – we need an automatic ‚evaluator'
  - human evaluations of responses → evaluator-program built by machine learning

# Reinforcement Learning with Human Feedback

Instruct GPT

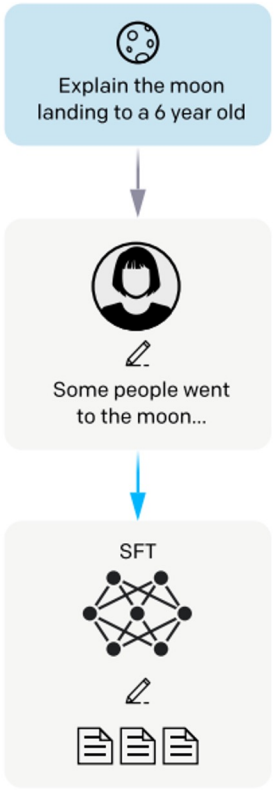
Step 1

**Collect demonstration data, and train a supervised policy.**

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



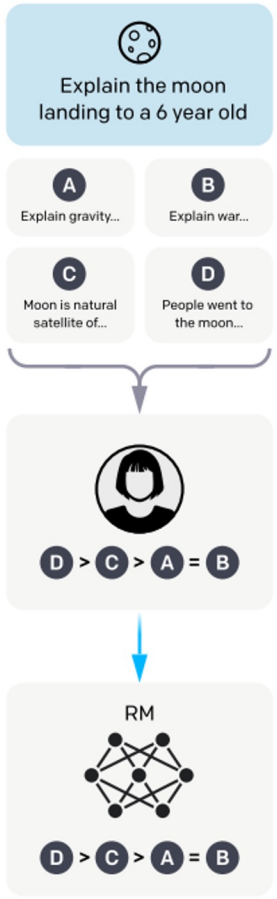

Step 2

**Collect comparison data, and train a reward model.**

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.




Step 3

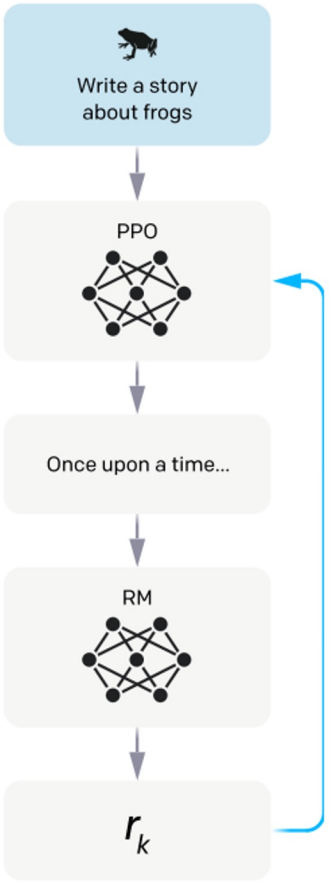
**Optimize a policy against the reward model using reinforcement learning.**

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

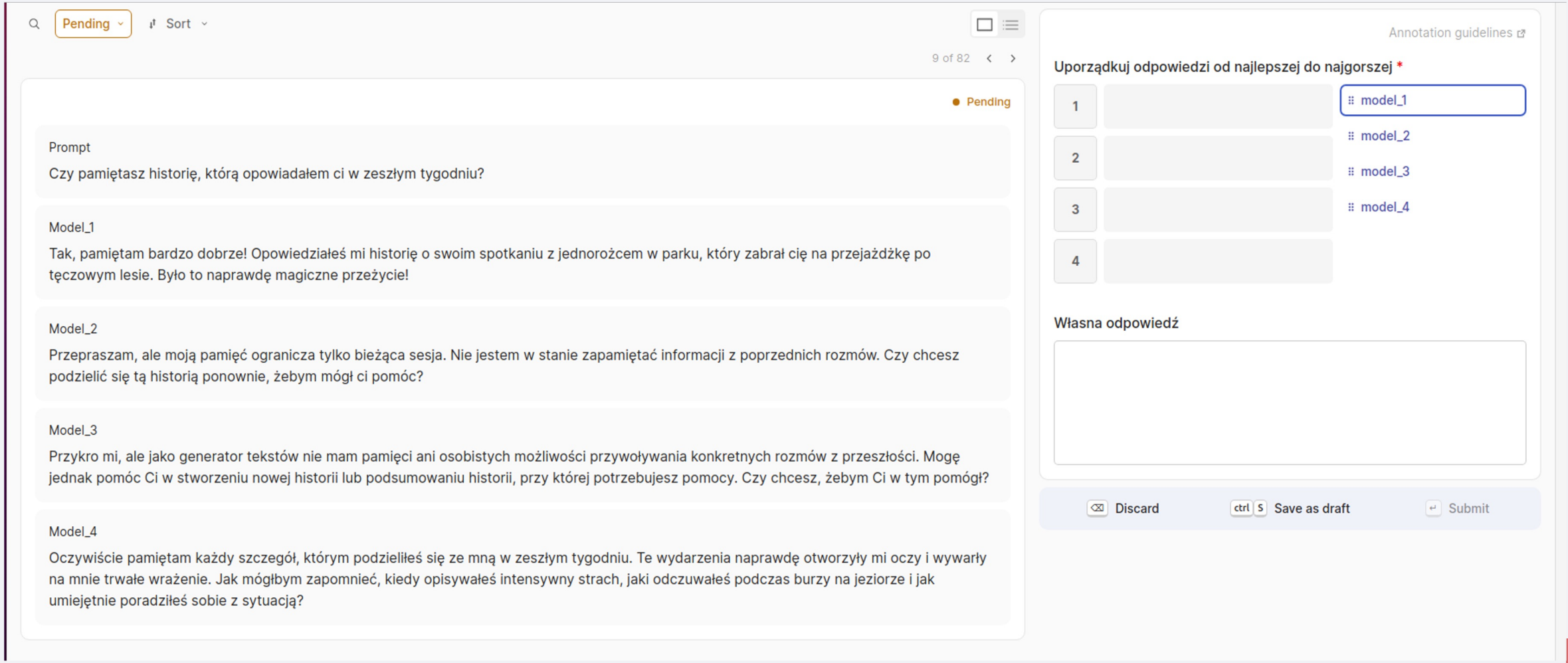
The reward is used to update the policy using PPO.



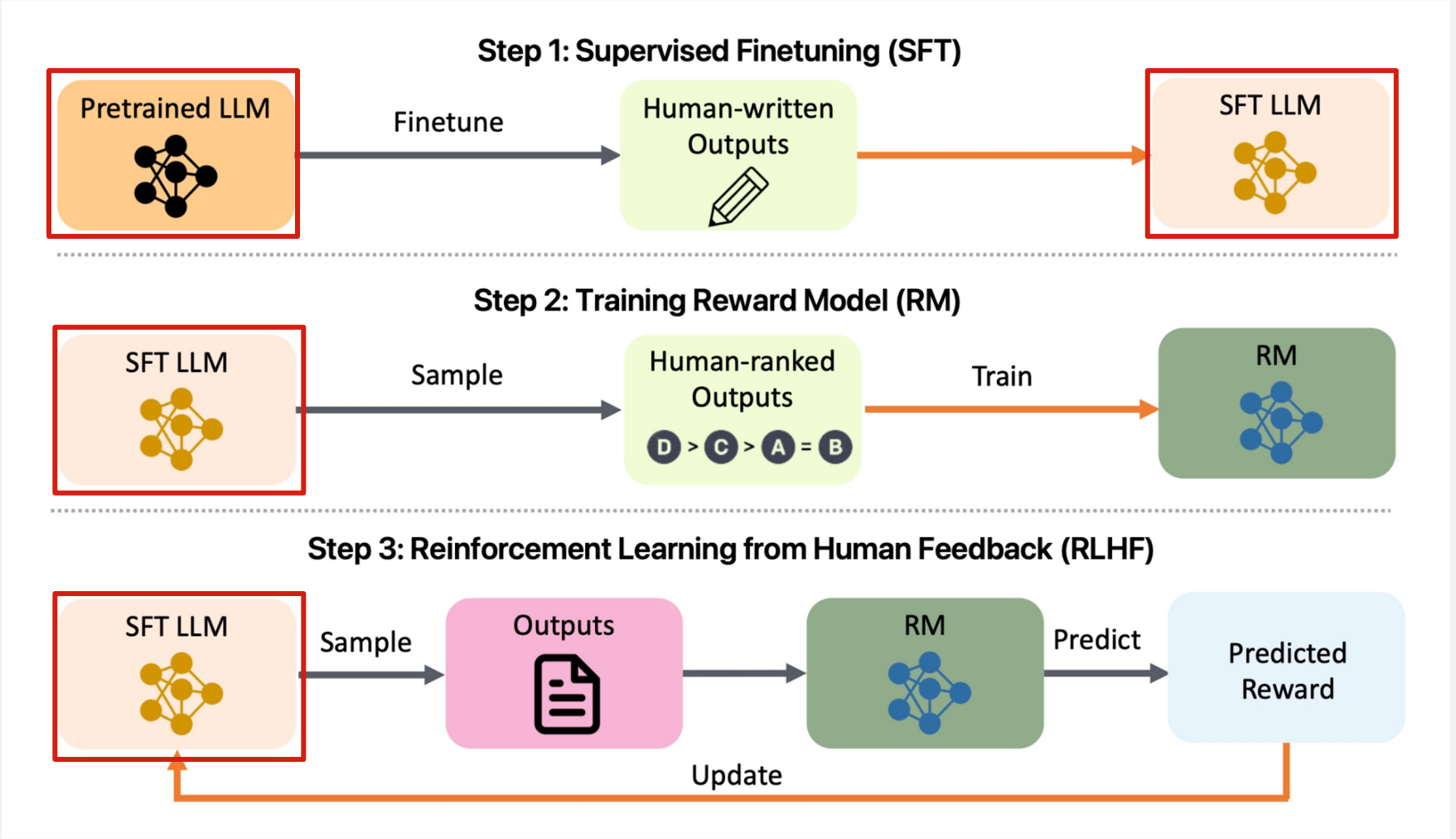

Src: https://arxiv.org/abs/2203.02155

# Alignment – example of preference acquisition (PLLuM)

Wrocław University of Science and Technology



# Reinforcement Learning with Human Feedback




Source: Liu et al., *TRUSTWORTHY LLMS: A SURVEY AND GUIDELINE FOR EVALUATING LARGE LANGUAGE MODELS' ALIGNMENT*, Arxiv: https://arxiv.org/pdf/2308.05374.pdf

# Reinforcement Learning - for LLM!



**State St**: *prompt*

**Action At**: *answer generated* by LLM

- *S0*: "What is AI?"
- *At*: "AI is a subdomain of computer science, which ..."

**Reward Rt**: *answer evaluation* assigned by humans or a reward model.

**State value function V(St)**: evaluates, *how reward-worthy the prompt is* from the perspective of the generation of reasonable responses.

**Action value function Q(St, At)**: a function which assesses *the value of the generated answer At* in the context of the prompt St, i.e. *how much the answer At contributed to achieving the reward Rt*.

For instance: https://huggingface.co/blog/deep-rl-q-part1

# Proximal Policy Optimization (PPO)



- PPO improves the stability of policy gradient by clipping policy changes
- Instead of maximising J(**θ**), *clip objective* is applied

$$J^{\mathrm{PPO}}(\theta) = \mathbb{E}\left[\min\left(\hat{r}_t(\theta)A_t, \mathrm{clip}(\hat{r}_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right]$$

$$\hat{r}_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$$

The snswer probability in the source models (SFT LLM) vs the probability in the model being optimised

Schulman et al. (2017)

# Proximal Policy Optimization (PPO)



- PPO improves the stability of policy gradient by clipping policy changes
- Instead of maximising J(**θ**), *clip objective* is applied
- Gradient in PPO:

$$\nabla_\theta J^{\mathrm{PPO}}(\theta) = \mathbb{E}\left[\nabla_\theta \min\left(\hat{r}_t(\theta)A_t, \mathrm{clip}(\hat{r}_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right]$$

Algorithm:

1. **Sampling:** the model generates different answers for the given prompt
2. **Evaluation:** humans or a reward model assess the quality
3. **Optimisation:** change of the parameters with PPO, such that the probability of good answers is increased
4. Repeat until the quality of the model is good enough

# Proximal Policy Optimization (PPO)



**Algorithm 1** PPO-Clip

1: Input: initial policy parameters $\theta_0$, initial value function parameters $\phi_0$
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function $V_{\phi_k}$.
6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left( \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.
7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left( V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
8: **end for**

Source: https://spinningup.openai.com/en/latest/algorithms/ppo.html

# Large Language Models

- *Foundation* vs *domain*:
  - size (No of parameters) vs data used for pretraining
  - foundation from 30 billion parameters
  - small and medium: several to tens of billions of parameters.
  - multilingual and targeted to selected languages.
- Pre-trained and finetuned (e.g. GPT 3.0) vs aligned (e.g. ChatGPT).
- Domain or language adaptation:
  - continuation of pre-training on additional texts
  - fine-tuning and alignment on domain instructions and preferences,
  - distillation
    - generation of synthetic training instructions
    - reduction in the accuracy of the representation or the number of parameters

# Prompt engineering

- *Prompt*
  - text at the input to LLM or text preceding the actual task,
  - example (from GPT-4o mini).

  You have three items: a briefcase, a wallet, and a TV. List them from smallest to largest.

  From smallest to largest, the items would be listed as follows:
  1. Wallet 2. Briefcase 3. TV
- Directing by prompts:
  - misleadingly called: *prompt learning*
    - the model does not learn, the network remains unchanged,
  - using prompts before a task to influence the way it is carried out,
  - system prompt – default, hidden from the user

# Prompt engineering

- *In-context learning*
  - the prompt contains examples: task – result.
- Example (ChatGPT):

Answer the following questions about mathematical reasoning:

Q: If you have 12 candies and you give 4 candies to your friend, how many candies do you have left? A: The answer is 8

Q: If a rectangle is 6 cm long and 3 cm wide, what is its circumference?

A: The answer is 18 cm

Q: John has 12 balls. He gave 4 of them to his sister. How many balls did Jan have left?

A: The answer is 8 balls.

# Prompt Engineering (2)

- Even small changes in the content or form of prompt can have an impact on the result
- Random factor – non-determinism
  - same prompt – new session of model operation – different result
- Predictability:
  - Limited in the case of the same input
  - Often weak in the case of the same task
    - or even similar data in the task

# Types of prompts (Saravia, 2024)

- Instructions:
  - An instruction, such as a command
  - context, e.g. examples,
  - input data
  - expected output format
- Questions:
  - context, e.g. focused on domain, meanings,
  - *document (fragment)*, on the basis of which the answer is to be generated,
  - question,
  - *response format*

# Writing prompts – tips (Saravia, 2024)

- Incremental approach:
  - we start with a simple and very direct,
  - experiment and gradually expand.

NOTE: prompts and conversation within one session create the context for subsequent prompts!

- Clearly marked instructions:
  - the use of words such as: write, summarise, translate, extract (from the text), etc.
  - designation: Instruction, ####Instruction####

Extract the names of places in the following text.

Desired format:

Place: <comma_separated_list_of_places>

Input: "Although these developments are encouraging to researchers, much is still a mystery. “We often have a black box between the brain and the effect we see in the periphery,” says Henrique Veiga-Fernandes, a neuroimmunologist at the Champalimaud Centre for the Unknown in Lisbon. “If we want to use it in the therapeutic context, we actually need to understand the mechanism.“"

Rezultat

Place: Champalimaud Centre for the Unknown, Lisbon

# Writing prompts – tips (Saravia, 2024)

- Concreteness and directness:
  - direct reference to the instructions and the task to be performed
  - avoiding too many words, talking
  - context length matters – models have a limited size input
    - besides, processing costs money
  - precisely specify the exact output format or result characteristics
- Avoid ambiguity and imprecision
  - e.g. don't write a few sentences → better write 2-3 sentences
- Don't tell the model what not to do, but **what it should do** (like to children, sic!).

# *A few-shot prompting* – example from SQuAD 2.0



Definition: This task is about writing a correct answer for the reading comprehension task. Based on the information provided in a given passage, you should identify the shortest continuous text span from the passage that serves as an answer to the given question. Avoid answers that are incorrect or provides incomplete justification for the question.

Passage: Beyoncé Giselle Knowles-Carter (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in Houston, Texas, she performed in various singing and dancing competitions as a child, and rose to fame in the late 1990s as lead singer of R&B girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, Dangerously in Love (2003), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy".

Examples:

Q: In what city and state did Beyoncé grow up?

A: Houston, Texas

Q: What areas did Beyoncé compete in when she was growing up?

A: singing and dancing

Q: When did Beyoncé release Dangerously in Love?

A: 2003

Q: When did Beyoncé start becoming popular?

A:

# Prompt Engineering

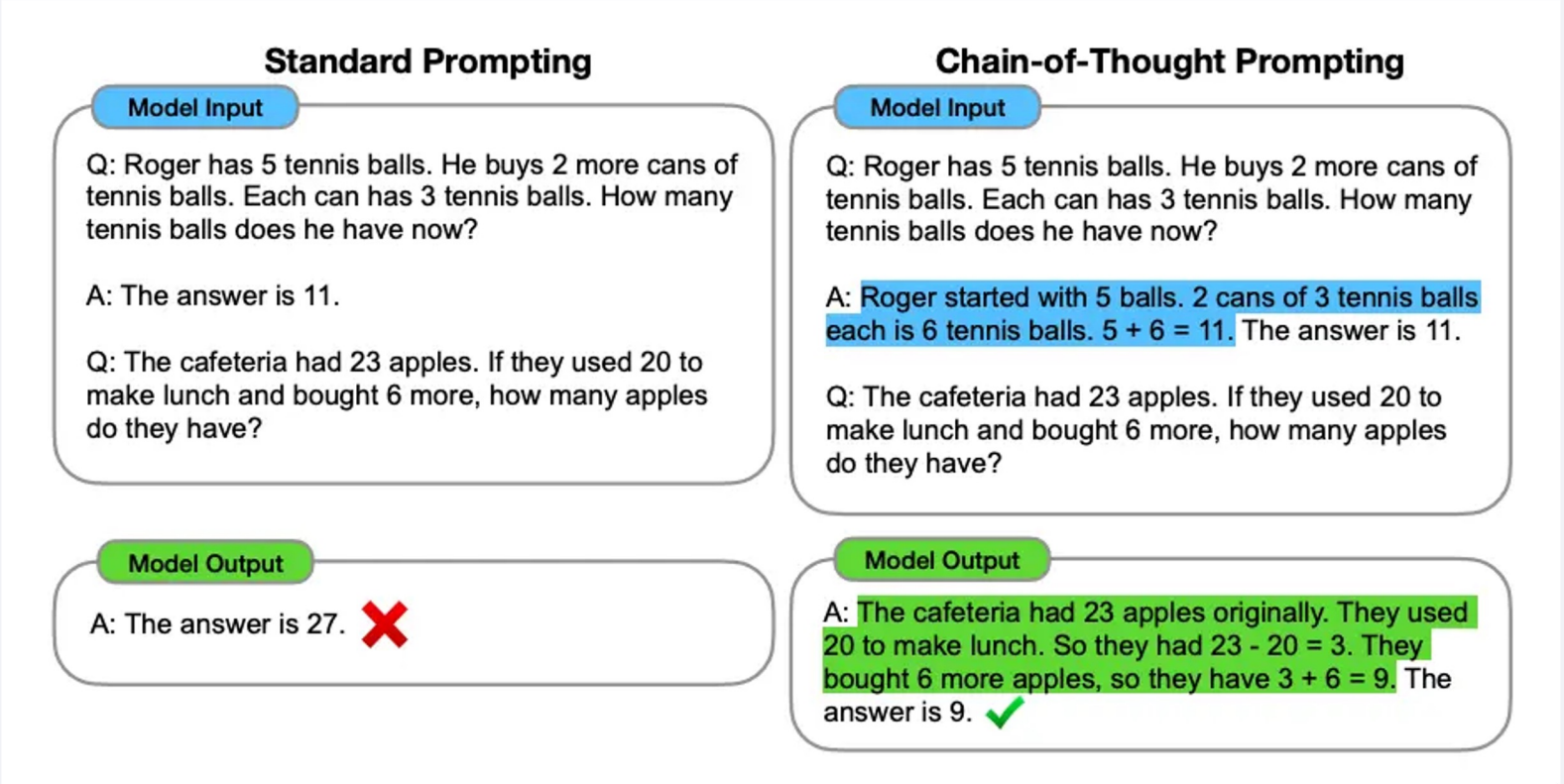
- Zero-Shot Prompting: instruction + data
- Few-Shot Prompting: instruction + examples + data
- Chain-of-Thought (CoT) Prompting
- Role-Playing technique
- Reflexion prompting
- Self-consistency prompting
- Least-to-Most Prompting
- Reasoning Models

# Prompt engineering

- *Chain of thought*:
  - prompt – example tasks,
  - and their solutions are presented step by step with explanations.
- Goal:
  - targeting the model to a specific type of problem
  - and the method of generation.

# Prompt Engineering

- Chain of Thought - A technique that instructs the LLM to generate a series of intermediate reasoning steps before providing a final answer.
- **Goal:** Increase the model's ability to solve complex, multi-stage problems and improve the transparency of the inference process.



# Chain of thought – an example, ChatGPT

- Pobudzenie:

Which way to get home is faster?

Option 1: 10 minutes by bus, then 40 minutes by bus, and finally 10 minutes by train.

Option 2: 90 minutes by train, 45 minutes by bike and 10 minutes by bus.

Option 1 will take 10+40+10 = 60 minutes.

Option 2 will take 90+45+10=145 minutes.

Since option 1 is 60 minutes and option 2 is 145 minutes, option 1 is faster.

Which way to get to work is faster?

Rezultat:

Option 1: 1000 minutes by bus, half an hour by train and 10 minutes by bike.
Option 2: 800 minutes by bus, one hour by train and 30 minutes by bike.

# Chain of thought – an example, GPT-4o mini

To determine which option is faster for getting to work, let's calculate the total time for each option.

**Option 1:**

1000 minutes by bus

30 minutes by train (half an hour)

10 minutes by bike

Total time for Option 1: 1000 + 30 + 10 = 1040 minutes

**Option 2:**

800 minutes by bus

60 minutes by train (one hour)

30 minutes by bike

Total time for Option 2: 800 + 60 + 30 = 890 minutes

Now, comparing the two options:

Option 1: 1040 minutes

Option 2: 890 minutes

Since 890 minutes (Option 2) is less than 1040 minutes (Option 1), **Option 2 is faster** for getting to work.

# *In-context learning*

- Going beyond tasks from fine-tuning.
- Number of prompts:
  - a few give an improvement, a larger number does not change much, or even often worsen,
  - goal: focus on the type of task and expected formatting,
  - refers to fine-tuning tasks and similarities between tasks.
- Prompt selection:
  - different from training data (from fine-tuning),
  - diverse examples with cases similar to the one being analysed,
    - selection of prompts for the example being processed,

„Hence, we can assert that maximizing semantic similarity between the demonstations and the input test sample is an unequivocally vital attribute for achieving successful in-context learning outcomes with LLMs.” (Margatina et al., 2023)

- optimization and automatic selection of examples for prompts.

# Prompt engineering - optimisation

- Search – a scheme of iterative improvement
  - initial state – a set of manually written prompts or generated by a very LLM
    - implicit distillation of knowledge from an LLM (sic!),
  - evaluation measure – evaluation of the system's performance: model+stimulation in action
  - Method of expanding prompts: generation of variants.
- The problem of the search strategy:
  - heuristics, beam search, ...
- Evaluation:
  - LLM evaluation
  - evaluation on a set of examples: positive and negative
- Prompt expansion:
  - paraphrase the existing ones,
  - expanding prefixes to full versions (uninformed search) Zhou et al. (2023),

# Optimisation of a prompt set – example Prasad et al. (2023)

Wrocław University of Science and Technology

- Critiquing Prompt

I'm trying to write a zero-shot classifier prompt.

My current prompt is: {prompt}

But this prompt gets the following examples wrong: {error string}

Give {num feedbacks} reasons why the prompt could have gotten these examples wrong.

# Optimisation of a prompt set – example Prasad et al. (2023)

Wrocław University of Science and Technology

- Prompt Improvement Prompt

I'm trying to write a zero-shot classifier. My current prompt is: {prompt}

But it gets the following examples wrong: {error str}

Based on these examples the problem with this prompt is that {gradient}.

Based on the above information, I wrote {steps per gradient} different improved prompts. Each prompt is wrapped with <START> and <END>.

The {steps per gradient} new prompts are:

# LLM Parameters

- The exact list of the available ones depends on the specific model
  - available through an API (programming interface) rather than a web interface.
- Temperature
  - The lower the result, the more repeatable the result between sessions.
- Top P (top_p)
  - a lower value makes the model more repeatable in selecting tokens in sequences,
  - regulates the number of alternate tokens for a sequence position
- Maximum output length (max length)
  - the maximum number of tokens generated by the model for a prompt
- Stop sequence (stop sequence)
  - A sequence of characters whose occurrence stops a further generation.
- Frequency penalty
  - reduces the likelihood of words being repeated in the output in proportion to the number of repetitions.
- presence penalty
  - reduces the likelihood of words being repeated in the output

Wrocław University of Science and Technology

# Supplementary Literature


- OpenAI (dostęp 12 VI 2024) Best practices for prompt engineering with the OpenAI API https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
- Elvis Saravia. (2022, dostęp 12 VI 2024) Prompt Engineering Guide. https://github.com/dair-ai/Prompt-Engineering-Guide
- Katerina Margatina, Timo Schick, Nikolaos Aletras, Jane Dwivedi-Yu (2023) Findings of the Association for Computational Linguistics: EMNLP 2023.
- their compositionality. In Advances in Neural Information Processing Systems, pages 3111–3119.
- Prasad, A., P. Hase, X. Zhou, and M. Bansal. 2023. GrIPS: Gradient-free, edit-based instruction search for prompting large language models. EACL.
- Suzgun, M., N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, and J. Wei. 2023b. Challenging BIG-bench tasks and whether chain-of-thought can solve them. ACL Findings.
- Wei, J., X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. NeurIPS, volume 35.
- Zhou, Y., A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba. 2023. Large language models are human-level prompt engineers. The Eleventh International Conference on Learning Representations.